LIFE OS

Complete Product Blueprint

A personal operating system for study, health, productivity, finances, learning and everyday life.

PROJECT DECISIONS
Scope: Personal use
Platform: Responsive web application + mobile experience (PWA-ready)
AI: Autonomous assistant with permission controls
Status: Product definition ready for implementation

CORE PRINCIPLE

One connected system—not a collection of isolated trackers. Every module should share relevant information so that plans are realistic, progress is measurable, and the AI can coordinate actions across the whole system.

Expanded from the original Life dashboard project document. Original requirements are retained and organized; recommended improvements and implementation decisions are added.

1. Product vision and scope

Life OS is a private, integrated workspace that helps one person plan and manage important areas of life from a single interface. It combines a calendar, task manager, study planner, health tracker, finance tracker, learning library, habits, goals and an AI operator.

Product principles

- **Integration first:** modules share data through common tasks, events, goals and activity records.
- **Realistic planning:** schedules account for existing commitments, energy, deadlines, buffers and prayer times.
- **Autonomy with safeguards:** the AI can perform routine actions, but sensitive or destructive actions require approval.
- **Useful feedback:** analytics explain what happened and suggest a practical adjustment, rather than judging the user.
- **Low-friction capture:** adding a task, expense, note, workout or idea should take only a few seconds.
- **Personal and private:** the system is designed for one owner, with strong access control and data portability.

Primary outcomes

Outcome	What success looks like
Clarity	At any moment, the user can see what matters today and why.
Consistency	Plans adapt to real completion patterns instead of repeatedly creating unrealistic schedules.
Academic progress	Lectures, revisions and exam deadlines are coordinated with available time.
Health awareness	Measurements, workouts and nutrition trends are visible over time.
Financial awareness	Spending, budgets and savings goals are understandable and actionable.
Continuous improvement	Daily and weekly reviews help improve future plans.

2. Information architecture

Use a small number of clear top-level destinations. Avoid making every feature a separate item in the main navigation.

Area	Main responsibilities
Dashboard	Daily overview, priorities, progress, focus timer, AI briefing and quick actions.
Calendar & Tasks	Unified agenda, tasks, events, recurring routines, time-blocking and reviews.
Study	Modules, lecture files, exams, study plans, revision scheduling, notes and performance.
Health & Fitness	Measurements, workout plans/logs, nutrition, water and health trends.
Finance	Transactions, categories, budgets, subscriptions and savings goals.
Library	Books, courses, podcasts, resources, progress and notes.
Goals & Habits	Long-term goals, projects, habits, milestones and streaks.
Rank & Achievements	Levels, quests, side quests, coins, rewards and achievement history.
Analytics	Cross-module trends, planning accuracy, consistency and progress.
Spoon Feeder AI	Global assistant available from every screen; can chat or open a full-screen workspace.

Navigation behavior

- Desktop: compact left sidebar with clear active states and a persistent quick-add action.
- Mobile: bottom navigation for Home, Calendar, Study, Library and More; secondary modules live under More.
- A floating Spoon Feeder AI control remains accessible without obscuring important content.
- Provide global search and a universal quick-add menu for task, event, note, expense, food, workout or resource.
- Support light and dark themes, responsive layouts and installable PWA behavior where supported.

3. Dashboard

The dashboard is the daily command center. It should answer: What should I do? What is important? How am I progressing?

Core dashboard components

- Today briefing: a short AI-generated summary of priorities, fixed commitments, deadlines and suggested focus.
- Timeline: today's scheduled events and time blocks, with clear free-time gaps.
- Tasks due today: priority, estimated duration, status and one-tap completion or rescheduling.
- Progress snapshot: study progress, habits, workouts, finance snapshot and goal milestones.
- Focus timer: Pomodoro timer linked to the current task or study session.
- Daily reflection: completion, obstacles, energy/mood (optional) and schedule feedback.
- Rank panel: level, XP, active quest and earned coins.
- Quick actions: add task, start focus session, log expense, record workout, add note.

Daily rating: make it fair and explainable

The original concept includes an AI rating based on achievement and commitment. Keep this as an optional summary, but do not reduce the whole day to one opaque score. Show separate dimensions and explain the calculation.

Metric	Meaning
Commitment	How much of the plan the user attempted or honored, with context for changes.
Completion	Proportion of planned tasks completed, weighted carefully by importance.
Consistency	Progress across days and weeks, not just one day's output.
Planning accuracy	How close estimated durations and planned workload were to reality.
Wellbeing balance	Whether the day allowed rest, breaks and personal commitments.

Do not punish illness, emergencies, changed priorities or tasks blocked by someone else. Let the user mark a task as blocked, deferred or intentionally dropped.

4. Calendar, tasks and scheduling

The calendar is the central scheduling engine for the entire product. All modules can create tasks or events that appear in the same calendar.

Important data distinction

Task: something to accomplish, with a status and possibly a deadline. Event: something that occupies a specific time, such as a lecture or appointment. A task can be completed in multiple scheduled work sessions.

Required capabilities

- Views: day, week, month, agenda and a focused today view.
- Tasks with priority, estimated duration, due date, category, status, subtasks and optional goal link.
- Events with start/end time, location or online link, reminders and recurrence.
- Recurring tasks and routines; university timetable and fixed commitments can be imported or entered once.
- AI time-blocking that considers deadlines, free time, estimated duration, task priority, energy preferences and buffers.
- Prayer-time-aware planning: prayer times are protected anchors, with configurable preparation/travel buffers and calculation method.
- Conflict detection and suggested alternatives when commitments overlap.
- Daily planning conversation and end-of-day review; weekly review for larger adjustments.
- Drag-and-drop rescheduling with a clear way to accept, reject or modify AI suggestions.

Scheduling workflow

- 1. Gather fixed events, recurring routines, deadlines and open tasks.
- 2. Ask the user about today's priorities, available time, energy and unusual constraints.
- 3. Place fixed commitments and prayer anchors first.
- 4. Schedule deadline-sensitive and high-priority tasks, then flexible work.
- 5. Add realistic buffers, breaks and travel time; avoid filling every minute.
- 6. Present the proposed plan and highlight trade-offs or unscheduled tasks.
- 7. Track what happened and ask for concise feedback at day's end.
- 8. Use feedback to improve future duration estimates and scheduling preferences.

5. Study management

The study module retains the original schedule-upload and spaced-repetition concept, and adds tools for reliable revision tracking and academic analytics.

Study setup

- Create academic terms, modules, lectures/topics and exam records.
- Upload module schedules, lecture lists, PDFs and exam deadlines; extract dates and topics for user confirmation.
- Allow manual correction of extracted information before it enters the plan.
- Set lecture importance, estimated study time, learning objectives and exam relevance.

AI study planning

- Generate a module-level plan from the lecture list, exam dates and available calendar time.
- Schedule first study sessions and follow-up revisions using spaced repetition.
- Use recall quality, confidence, difficulty and quiz performance to adjust the next review date.
- Consider an FSRS-style scheduling algorithm for flashcards/review items rather than relying only on a fixed interval.
- Rebuild the plan when lectures are delayed, exams move or availability changes.

After-study feedback

- Record what was studied, duration, completion status and optional notes.
- Ask for a quick recall rating and perceived difficulty; avoid making feedback burdensome.
- Optionally run a short active-recall quiz to obtain a more objective signal.
- Schedule the next revision and show why that date was selected.
- Support lecture notes, highlights, attachments, flashcards and question sets.

Study analytics

- Lecture coverage and completion before exam deadlines.
- Revision adherence and overdue reviews.
- Recall/quiz performance over time and topic-level weak areas.
- Planned versus actual study time and study-session consistency.
- Forecast whether the remaining workload fits the time before an exam.

AI study tools

Spoon Feeder AI may summarize uploaded material, generate flashcards, create practice questions, explain difficult concepts and convert lecture content into a study plan. Clearly distinguish source-grounded outputs from additional explanations, and preserve references to the source file where possible.

6. Health and fitness

The health area has three original pillars—body measurements, workout tracking and diet tracking—with supporting routines and trend analysis.

6.1 Body measurements

- Record weight and user-selected measurements such as waist, chest, arms or other metrics.
- Show time-series graphs, date comparisons and progress toward optional targets.
- Allow private notes and measurement reminders; do not infer measurements.

6.2 Workout tracker

- Create workout plans manually or ask AI to draft a plan based on goals, equipment, experience, schedule and preferences.
- Store exercises, sets, reps, load/weight, rest periods and optional RPE or notes.
- Compare performance between sessions and show volume, personal records and consistency.
- Allow exercise substitutions and record missed or modified sessions.
- Calendar integration: workouts become scheduled events or tasks.

6.3 Nutrition tracker

- Log meals and food portions; calculate estimated calories and macronutrients from a food database.
- Allow manual food entry and corrections when database values are incomplete.
- Support user-defined nutrition targets and optional meal plans.
- Daily review should identify patterns and offer practical suggestions, not label the user or a day as simply 'good' or 'bad'.
- Clearly mark estimates and avoid presenting nutrition calculations as medical advice.

Additional health improvements

- Optional water intake and sleep/activity tracking.
- Privacy-first storage and controls for exporting or deleting health records.
- AI-generated plans must be reviewable and adjustable; avoid unsafe extreme recommendations.

7. Personal finance tracker

Expand the original money tracker into a simple personal finance workspace. Keep data entry fast and make the dashboard useful without requiring complex accounting.

Core features

- Accounts or wallets with opening balances and optional cash/bank separation.
- Income and expense transactions with amount, date, category, account and notes.
- Custom categories, recurring transactions and subscription tracking.
- Monthly budgets by category, spending summaries and alerts when approaching a limit.
- Savings goals with target amount, target date and progress.
- Search, filters, monthly comparisons and CSV export.

Data rules

- Store monetary values in exact decimal or integer minor units, not floating-point approximations.
- Support the user's preferred currency and allow future multi-currency support if needed.
- Do not let AI silently create, edit or delete financial records without clear confirmation for consequential actions.

8. Knowledge library: books, courses, podcasts and notes

Retain the original reading/listening tracker and make it a unified personal learning library.

Library item types

- Books, courses, podcasts, articles, videos and other learning resources.
- Statuses such as wishlist, planned, in progress, paused and completed.
- Progress tracking by pages, chapters, lessons, episodes or percentage.
- Notes, key takeaways, tags, ratings and links/attachments.
- Search and filters by topic, status, format and date.

Unified notes and knowledge

- Notes can be linked to a lecture, book, course, task, goal or project.
- Use tags and global search to retrieve information across the library.
- Allow quick capture of an idea without deciding its final location immediately.
- AI can summarize or search the user's own notes, while identifying the notes used as sources.

9. Goals, projects and habits

Goals and projects

- Create long-term goals with a reason, target date, milestones and measurable indicators.
- Break goals into projects, tasks and recurring actions.
- Link tasks and habits to goals so daily activity contributes to visible progress.
- Show stalled goals and recommend a next small action.

Habit tracker

- Create daily, weekly or custom-frequency habits with flexible targets.
- Track completion, partial completion, skips and reasons without shaming.
- Show streaks, consistency over time and calendar heatmaps.
- Allow habits to become calendar blocks when they need a specific time.

10. Rank, quests and rewards

Gamification should motivate meaningful behavior, not encourage overwork or reward meaningless task volume.

Original mechanics retained

- Ranks and levels based on task completion, commitment and consistency.
- Main quests and optional side quests.
- Coins earned through defined actions and spent on user-configured rewards.
- Achievements, streak milestones and progress history.

Recommended improvements

- Reward consistency, learning effort, healthy routines and goal progress—not only the number of completed tasks.
- Use weekly quests and flexible goals so one difficult day does not destroy progress.
- Let the user define meaningful rewards and coin costs.
- Make XP rules transparent and avoid double-counting the same activity across modules.
- Include rest and recovery as legitimate parts of a balanced system.

11. Spoon Feeder AI: autonomous assistant

The assistant is the main interaction layer across Life OS. It should be accessible as a floating spoon icon, open in a compact chat panel, and expand to a full-screen workspace.

Capabilities

- Understand natural-language requests and retrieve relevant information from connected modules.
- Create, update, reschedule and complete tasks; organize calendars; draft study plans; log workouts, meals and expenses.
- Summarize uploaded study material and generate notes, flashcards and questions.
- Prepare daily briefings, end-of-day reviews and weekly summaries.
- Proactively identify conflicts, overdue revisions, approaching deadlines and patterns worth attention.
- Explain what it changed and provide an action history.

Autonomy and permission model

Level	Examples	Rule
Read	View tasks, calendar, study progress, goals and analytics.	Allowed within the user's permissions.
Routine write	Create a draft task, log a completed focus session, update progress.	May execute automatically if user has enabled it; always log.
Consequential	Move major deadlines, change a full study plan, alter budgets or recurring schedules.	Show a preview and request confirmation.

Level	Examples	Rule
Sensitive/destructive	Delete records, export private data, change security settings or perform irreversible actions.	Explicit approval required; offer undo where possible.

Required AI safeguards

- Use defined application tools and validated operations; never give the model unrestricted database or SQL access.
- Validate every action server-side and enforce ownership/permissions.
- Use a preview/approval step for consequential changes and destructive actions.
- Maintain an AI action log with request, action, affected records, timestamp and outcome.
- Support undo or reversal for eligible actions.
- Ask a clarifying question when the target or intent is ambiguous; never guess which record to change.
- Clearly identify generated content and keep source references for summaries where possible.

12. Cross-module integration

Integration is the defining feature of Life OS. Each module should publish meaningful events so other modules can respond without tightly coupling every feature to every other feature.

Examples of connected workflows

When this happens	Life OS response
A new exam date is added	Study planner estimates workload and places revision sessions into available calendar time.
A lecture is marked studied	Study history updates, next revision is calculated, analytics refresh and a task may be completed.
A workout is planned	It appears in the calendar and can be considered by the daily scheduler.
A workout is completed	Workout history and progress charts update; a related quest may progress.
A habit is completed	Habit history, goal progress and eligible rewards update.
An expense is recorded	Budget totals and finance dashboard update.
A goal milestone is reached	Goal progress, dashboard and achievement system update.
A task is rescheduled	Calendar and daily briefing update; dependent plans are re-evaluated if necessary.

Recommended event types

- task.created / task.completed / task.rescheduled
- calendar_event.created / calendar_event.updated
- lecture.studied / revision.completed / revision.due
- workout.completed / measurement.recorded / food_entry.created
- expense.created / budget.threshold_reached
- habit.completed / goal.milestone_reached / library_item.completed

Technical rule: event handlers should be idempotent. If the same event is processed twice, it must not create duplicate rewards, revisions or records.

13. Analytics, reviews and notifications

Analytics should help the user make decisions. Avoid dashboards full of numbers that do not lead to an action.

Analytics principles

- Show trends over meaningful time ranges: week, month, term and custom range.
- Separate planned work from completed work and distinguish fixed events from flexible tasks.
- Explain metric definitions and allow drill-down to the records behind a chart.
- Use neutral language; show context and possible causes rather than blame.
- Do not treat every module as equally important every day.

Review cadence

- Morning: short planning conversation to confirm priorities and constraints.
- Evening: review completion, what changed, what blocked progress and whether estimates were realistic.
- Weekly: review goals, study workload, habits, health activity and spending; suggest a small number of adjustments.

Notifications

- Reminders for time-sensitive events, due revisions, deadlines and user-selected habits.
- Quiet hours and per-category notification controls.
- Avoid duplicate reminders when multiple modules refer to the same event.
- Prioritize actionable notifications and allow snooze, dismiss or reschedule.

14. Technical architecture

For a personal project, start with a modular monolith: one application and one database, organized into clear domains. This is easier to build and maintain than microservices while leaving room to scale later.

Layer	Recommended approach
Frontend	Next.js + TypeScript; responsive UI; Tailwind CSS and a reusable component system such as shadcn/ui.
Backend	Next.js server actions/API routes or a dedicated service layer with validated business operations.
Database	PostgreSQL, with Supabase as a practical managed option.
Authentication	Supabase Auth; private single-user account initially; secure session handling.
File storage	Private object storage for lecture PDFs, images and attachments; signed access where appropriate.
AI layer	Tool-calling agent service with typed tools, permission checks, approvals and action logs.
Background jobs	Scheduled and event-driven jobs for reminders, recurring tasks, revision due dates and summaries.
Charts and calendar	A chart library such as Recharts and a calendar component such as FullCalendar.
Deployment	Vercel for the web application and Supabase for managed backend services; configure backups.
Mobile experience	Responsive web app first, then PWA installation and optional native wrapper if needed.

Architecture rules

- Keep business logic in reusable server-side services, not only in UI components.
- Use typed interfaces and schema validation for every write operation.
- Keep module boundaries clear: Study, Calendar, Health, Finance, Library, Goals, Gamification and AI.
- Use background jobs for work that should not depend on a browser tab remaining open.
- Store timestamps consistently in UTC and retain the user's timezone for display and scheduling.

15. Core data model

The following is a recommended starting schema. It is a domain map, not a final migration script; exact columns and relationships should be finalized during implementation.

Domain	Main entities / tables
Core	profiles, user_settings, tasks, calendar_events, recurring_rules, goals, projects, activity_logs, notifications
Study	modules, lectures, lecture_files, lecture_notes, study_sessions, revision_records, flashcards, flashcard_reviews, exam_dates, study_plans
Health	body_measurements, workout_plans, exercises, workout_sessions, exercise_sets, food_entries, food_database, daily_nutrition, water_logs
Finance	accounts, transactions, categories, budgets, savings_goals, subscriptions
Knowledge	library_items, library_progress, knowledge_notes, knowledge_tags, knowledge_resources
Gamification	user_stats, quests, quest_progress, achievements, rewards, reward_redemptions
AI	ai_conversations, ai_messages, ai_memories, ai_action_logs, ai_approvals, ai_plans, ai_feedback

Data conventions

- Use UUID primary keys and explicit foreign-key relationships.
- Include created_at and updated_at on important records.
- Use explicit status fields rather than relying on ambiguous null values.
- Use soft deletion for important user records where recovery is useful.
- Keep money in exact decimal/minor-unit representation.
- Record timezone and source attribution where relevant.
- Add indexes for common filters such as date, status, module, category and owner.
- Use database transactions for multi-step operations that must succeed together.

16. Privacy, security and reliability

Because Life OS may contain academic files, health information, personal finances and private routines, security must be part of the foundation rather than a later feature.

Minimum requirements

- Authentication and strict row-level security so only the owner can access their records.
- Private file storage with access controls; do not expose public URLs for private documents.
- Server-side validation and authorization for every write operation.
- Secure transport (HTTPS), protected secrets and no API keys in frontend code.
- Limit what is sent to external AI services; send only the context required for the task.
- Provide data export and account-level deletion workflows.
- Automated backups and a tested restore process.
- Audit important AI and user actions, especially edits to schedules, finance and health records.
- Use error monitoring and clear recovery behavior when background jobs fail.

Reliability considerations

- The calendar should remain usable if the AI service is temporarily unavailable.
- Allow manual task creation and editing at all times.
- Retry background jobs safely without duplicating actions.
- Show when data was last synchronized or calculated.
- Do not silently discard failed imports; show errors and allow correction.

17. Development roadmap

Build a reliable core before adding every advanced feature. The phases below prioritize a usable personal system and reduce the risk of building many disconnected modules.

Phase	Build	Exit criteria
0. Foundation	Project setup, authentication, database, design system, settings, navigation and backups.	Secure app shell; data model and migrations; usable on mobile and desktop.
1. Core productivity	Dashboard, tasks, calendar, recurring rules, prayer-time anchors, daily planning and reviews.	A normal day can be planned and tracked without AI.
2. Study	Modules, lecture uploads, exams, study plans, revision scheduling, notes and analytics.	A module can be planned through to exam completion.
3. Habits and goals	Goals, projects, habits, milestones and basic progress analytics.	Daily actions connect to longer-term goals.
4. AI assistant	Chat, retrieval, tool calling, routine actions, approvals, audit log and undo.	AI can safely perform a defined set of useful actions.
5. Health and finance	Measurements, workouts, nutrition, expenses, budgets and savings goals.	Health and money data can be logged and reviewed.
6. Library and gamification	Books/courses/podcasts, unified notes, quests, levels, coins and rewards.	Progress and rewards reflect activity across modules.
7. Polish	PWA improvements, notifications, offline-friendly behavior, analytics refinements and testing.	Stable daily use across desktop and mobile.

Recommended first usable release (MVP)

- Dashboard with today's priorities and a simple focus timer.
- Calendar and task manager with recurring tasks and prayer-time anchors.
- Study module with lectures, exam dates, a basic study plan and revision tracking.
- Habits and daily review.
- Spoon Feeder AI that can read relevant data, draft plans and execute a small set of approved routine actions.

Do not start by building every chart, every reward mechanic or fully autonomous AI behavior. First prove that the core calendar, tasks and study workflow are reliable.

18. First implementation checklist

Use this as the practical handoff from product definition to development.

Before coding

- Confirm the final navigation and dashboard wireframes.
- Define the first-release user journeys: add a task, plan a day, add a lecture, schedule a revision and complete a review.
- Finalize the core database schema and ownership rules.
- Define prayer-time calculation method, location handling and manual override behavior.
- Choose the initial AI model/provider and define the first typed tool contracts.
- Define which AI actions are automatic, approval-based or prohibited.

Build and test

- Create authentication, settings and responsive application shell.
- Implement tasks and calendar before AI time-blocking.
- Test recurring events, timezone changes, daylight-saving transitions where applicable and prayer-time updates.
- Test duplicate background events and repeated AI tool calls.
- Test mobile layouts, keyboard access, loading states and error recovery.
- Add database backups and verify restore before storing important personal records.

Definition of done for the first release

- The user can plan and adjust a day manually.
- The calendar combines tasks, study sessions and fixed events.
- Study sessions and revisions can be tracked against exam dates.
- The app works comfortably on phone and desktop.
- AI can explain a proposed plan and perform only the actions permitted by the configured policy.
- All important changes are saved, auditable and recoverable where possible.

Final product statement

Life OS is a personal, AI-assisted operating system that connects the user's commitments, learning, health, finances and goals. Its value comes not from having the most features, but from making those features work together in a reliable, private and practical way.